Projeto MenuFlex - Atividade Somativa

Contextualização

A empresa MenuFlex Tecnologia presta serviços de desenvolvimento de sistemas web para pequenos negócios. Um de seus clientes, o restaurante "Cantina Bella Vita", solicitou a criação de um sistema para gerenciamento digital de seu cardápio.

A empresa já decidiu que o sistema será desenvolvido em arquitetura web, utilizando API REST para o back-end.

Antes do início do desenvolvimento completo, a equipe técnica precisa validar a modelagem, as decisões tecnológicas e a estrutura da API, garantindo que o projeto seja escalável e organizado.

Você foi designado para realizar essa análise técnica inicial.

Etapa A – Modelagem Orientada a Objetos (Diagrama de Classe)

1 - Elabore o **diagrama de classe** da entidade **Produto**, considerando um sistema de cardápio digital.

O diagrama deve conter, no mínimo:

- Atributos e métodos essenciais;
- Tipos de dados;

Etapa B – Persistência e Base de Dados

1 - Explique como a entidade **Produto** seria armazenada em um banco de dados.

RESPOSTA:
Ela iria fazer a conexão com o mySQL, onde seria armazenada no arquivo doteenv, onde irá ficar todos os métodos da entidade produto e depois passadas pelo arquivo.

Sua resposta deve abordar:

- Tipo de banco de dados mais adequado (relacional ou não relacional);
  RESPOSTA:
  Relacional

- Justificativa da escolha;
  RESPOSTA:
  Pois ele pega todas as informações e faz elas se conectarem, buscando uma só razão. Já o não relacional, embaralha tudo, deixando bagunçado, o que não seria uma alternativa para escolha nesse exercício.

- Forma de interação entre a API e o banco de dados.
  RESPOSTA:
  A forma de interação entre api e banco de dados seria permitida por meio do pool de conexões, localizado no database.js. Armazena todo o processo inserido dentro dessa const pool, com a ajuda do env.

Resposta:

2 - Veja o arquivo database.js:

```
const pool = mysql.createPool({
host: 'localhost',
user: 'root',
password: '',
database: 'bellavita',
porta: 3306
})
```

Por que essa conexão não vai funcionar? Explique qual é o erro e como corrigir.

RESPOSTA:
Não iria funcionar pela falta de utilização do env dentro do código. Pois o mesmo precisa guardar as informações credenciais para enviar dentro do arquivo .env, para conseguir dar continuidade no ambiente do projeto do código.

3 - Observe o código:

```
app.put('/produtos/:id', async (req, res) => {
    const { id } = req.params
    const dados = req.body

    await queryAsync("UPDATE produto SET ? WHERE id = id", [dados])
    res.send("Atualizado")
})
```

Por que esse UPDATE não irá atualizar nenhum produto?

Resposta:
O uso do app.put, não iria funcionar corretamente, pela falta de informações necessárias para o atualizar algo. Primeiro deveria estar antes do const um "try" onde guardaria a const do {id} que é alto inclemente, e outro const que estaria toda a sessão que irá atualizar, como "nome, preco, disponibilidade e etc"

Etapa C – Arquitetura da API REST

1 - Descreva a estrutura básica de uma **API REST** para gerenciamento de produtos.
Inclua em sua resposta:

- Endpoints necessários para as operações de CRUD;
- Métodos HTTP associados a cada operação;
- Organização lógica das rotas.

ENDPOINTS NECESSÁRIOS PARA OPERAÇÃO DA CRUD:
Os Endpoints necessários para as operações de Crud seria a Pool de conexões onde iria fazer a conexão com o MySQL, para buscar as credenciais/informações, criados dentro do banco, o próximo passo seria buscar a Porta que vamos trabalhar, criando um listen(teste de conexão) para conseguirmos receber o resultado dessa relação, sempre observando a habilitação do nosso app e pool(database).

METODOS HTTP:
Os principais métodos http utilizados foram GET: /produtos, /produtos/id.
POST: post/produtos. PUT: /produtos/:id DELETE: /produtos/:id.

ORGANIZAÇÃO LÓGICA DE ROTAS:

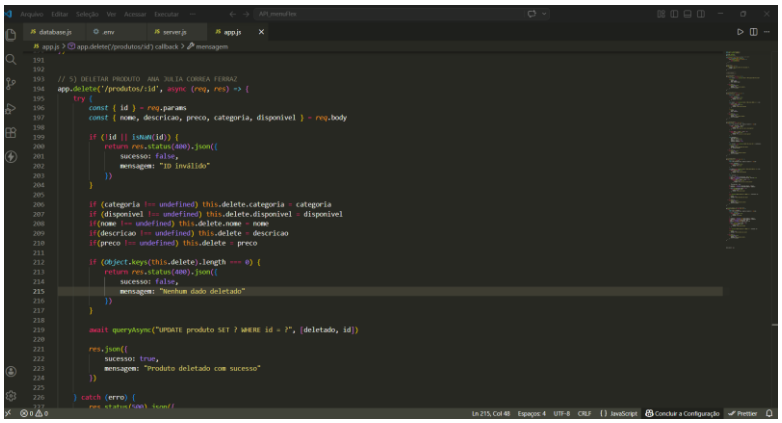
As rotas lógicas seria o GET/produtos (onde tem a função de listar todos os produtos), GET/produtos/id (Buscar qualquer produto pelo um único ID), POST/produtos (Cadastra um produto) PUT/produtos/:id (tem a função de atualizar um produto já cadastrado, DELETE/produtos (deleta um produto já cadastrado)

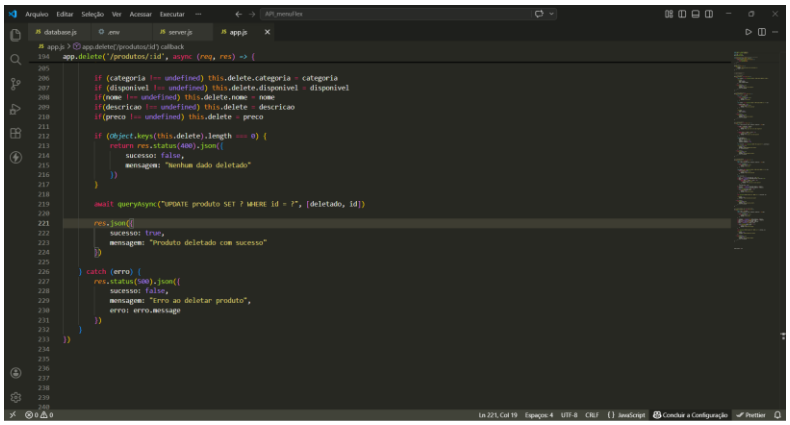
2 - Analise a rota abaixo:

```
app.get('/produtos/id', async (req, res) => {
    const id = req.params.id
    const produto = await queryAsync("SELECT * FROM produto WHERE id = ?", [id])
    res.json(produto)
})
```

Por que essa rota nunca vai funcionar ao buscar um produto pelo ID?
Não iria funcionar pela falta de comunição dos códigos entre si. Faltando a adequação do uso do catch para se comunicar com o erro. Casos de produto igual a 0 ou id invalido.

3 – Desenvolva o método DELETE de produtos.





Etapa D – Frameworks e Ambiente de Desenvolvimento

1 - Explique por que o uso do **Node.js com Express** é adequado para o desenvolvimento dessa API.
Sua resposta deve considerar:

- Características do Node.js;
- Papel do framework Express;
- Benefícios para aplicações web.

***RESPOSTA*:**
O uso do node.js com express são indicados por funcionarem como uma biblioteca de dados, onde vai te ajudar com benefícios de armazenamento do seu projeto, os dois juntos têm suas principais funções sendo corrigir e reparar questões para prestar serviço dentro do ambiente de seu projeto.

2 – Descreva, de forma clara e organizada, as etapas iniciais para criar um projeto web em Node.js com persistência de dados em MySQL. Explique também como você estruturaria a organização do projeto (pastas e arquivos principais).

***RESPOSTA*:**
A etapa principais para a criação seria começar criando as pastas "config", onde estaria localizado o database que faz a interação com o MySQL. Logo em seguida utilizando o node init -y, para a CRIAÇÃO DA API, depois entrar dentro da nossa API, e começar a conexão dos dados como app, database,server.js e etc.

3 – Para que serve o arquivo *.env* e por que não devemos subi-lo no github?
***RESPOSTA*:**
Não pode subir ao github, pois ele é um arquivo que é armazenado todas suas informações confidenciais, se você subir ao github, todos irão conseguir ver suas informações.